

Relatório Técnico Individual: Planejador de Entregas por Drones em Cidade Inteligente

Autor: Felipe Mariano Monti

Data: 06 de Maio de 2026

1. Introdução

Este relatório técnico descreve o desenvolvimento de um sistema de planejamento de entregas por drones para uma cidade inteligente, abordando os requisitos definidos no documento "Alg2-Projeto.pdf" [1]. O objetivo principal é otimizar a seleção de entregas para um drone, considerando restrições de peso, bateria, tempo e quantidade máxima de entregas, visando maximizar a recompensa total. Para isso, foram implementadas e comparadas duas abordagens algorítmicas: um algoritmo guloso e o algoritmo de backtracking.

2. Descrição do Problema

O problema consiste em selecionar um subconjunto de entregas disponíveis que um drone pode realizar, respeitando suas capacidades máximas de peso, bateria, tempo de voo e número de entregas. Cada entrega possui um ID, peso, prioridade, recompensa, consumo de bateria, tempo de duração e distância. O critério de otimização adotado é a maximização da recompensa total das entregas selecionadas.

Restrições do Drone:

- **Peso Máximo:** Capacidade máxima de carga que o drone pode transportar.
- **Bateria Máxima:** Capacidade total da bateria do drone, que se esgota com o tempo de voo e a distância percorrida.
- **Tempo Máximo:** Duração máxima da missão do drone.
- **Número Máximo de Entregas:** Quantidade máxima de entregas que o drone pode realizar em uma única missão.

3. Metodologia e Abordagens Algorítmicas

Para resolver o problema, foram implementadas duas abordagens:

3.1. Algoritmo Guloso (Heurística de Eficiência)

O algoritmo guloso busca uma solução localmente ótima em cada etapa, na esperança de encontrar uma solução globalmente ótima. Neste projeto, a heurística utilizada para ordenar as entregas foi a **recompensa por unidade de peso e bateria**. As entregas são ordenadas de forma decrescente com base na razão $\text{recompensa} / (\text{peso} + \text{bateria}/10)$. O algoritmo então seleciona as entregas nessa ordem, desde que as restrições do drone não sejam violadas. Esta heurística visa priorizar entregas que oferecem maior retorno em relação aos recursos consumidos.

3.2. Algoritmo de Backtracking (Busca Exaustiva)

O algoritmo de backtracking é uma técnica de busca exaustiva que explora todas as combinações possíveis de entregas. Ele constrói uma solução passo a passo e, se em algum momento a solução parcial se torna inviável ou não promissora, ele "volta atrás" (backtrack) para tentar outra alternativa. Esta abordagem garante a **solução ótima** para o problema, mas pode ter um custo computacional elevado para instâncias maiores, devido à sua natureza exponencial.

4. Implementação

O projeto foi implementado em C++ e segue uma arquitetura orientada a objetos, conforme solicitado. As principais classes e estruturas de dados são:

- **Entrega** : Representa uma entrega individual, com atributos como `id`, `peso`, `prioridade`, `recompensa`, `bateria`, `tempo` e `distancia`. Possui métodos `getter` para acesso aos atributos.
- **Drone** : Representa o drone, com atributos para suas capacidades máximas: `maxPeso`, `maxBateria`, `maxTempo` e `maxEntregas`. Também possui métodos `getter`.
- **Resultado** : Uma estrutura para armazenar os resultados de cada algoritmo, incluindo o método utilizado, as entregas selecionadas, o peso total, bateria total, tempo total, recompensa total e tempo de execução.

4.1. Leitura de Dados

O programa oferece flexibilidade na entrada de dados:

- **Dados Fixos no Código**: Casos de teste pré-definidos para facilitar a demonstração e teste rápido.
- **Leitura de Arquivo CSV**: Uma função `carregarCSV(string filename)` foi implementada para ler dados de entregas a partir de um arquivo `.csv`. O arquivo deve conter um cabeçalho

e os dados das entregas em cada linha, separados por vírgulas, na ordem:

```
id,peso,prioridade,recompensa,bateria,tempo,distancia .
```

4.2. Estrutura do Código

O código é modularizado em funções para cada algoritmo (`resolverGuloso` , `resolverBacktracking`) e para a impressão dos resultados (`imprimirTabela`) e comparação (`comparar`). A função `main` implementa um menu interativo que permite ao usuário escolher a fonte dos dados e executar os algoritmos, exibindo os resultados e a comparação final.

4.3. Tratamento de Solução Inviável

O sistema verifica se alguma solução foi encontrada. Caso contrário (por exemplo, se nenhuma entrega puder ser selecionada devido às restrições do drone), uma mensagem de alerta é exibida, indicando que "Nenhuma solução viável encontrada para as restrições dadas".

4.4. Formatação de Saída

A saída no console é formatada utilizando a biblioteca `<iomanip>` para garantir o alinhamento e a legibilidade dos dados, simulando um painel de controle logístico.

5. Comparação de Resultados

O programa compara o desempenho do algoritmo guloso e do backtracking em termos de recompensa total, número de entregas realizadas e tempo de execução. Uma tabela comparativa é exibida no console, seguida de uma análise textual destacando qual abordagem obteve melhor resultado e a diferença de tempo de execução.

6. Como Compilar e Executar

Para compilar o código C++, utilize um compilador como g++:

```
Bash
```

```
g++ -std=c++11 -o projeto_drones projeto_drones.cpp
```

Para executar o programa:

```
Bash
```

```
./projeto_drones
```

O programa apresentará um menu interativo no console, permitindo ao usuário escolher entre carregar dados de um arquivo CSV ou utilizar casos de teste pré-definidos.

7. Conclusão

Este projeto demonstra a aplicação de algoritmos de otimização para resolver o problema de planejamento de entregas por drones. A comparação entre o algoritmo guloso e o backtracking ilustra o trade-off entre a busca pela solução ótima e a eficiência computacional. O algoritmo guloso, embora não garanta a otimalidade, oferece uma solução rápida e geralmente boa, enquanto o backtracking garante a solução ótima, mas com um custo computacional maior, especialmente para instâncias maiores do problema. A implementação em C++ com classes e a capacidade de ler dados de CSV tornam o sistema robusto e flexível para diferentes cenários.